

OilSpill-Attribution — System Architecture & Integration Reference

Intended End-to-End Pipeline: Image → ML Inference → Drift Backtracking → AIS Processing → Vessel Matching → Composite Scoring → Most Likely Source Vessel → Frontend

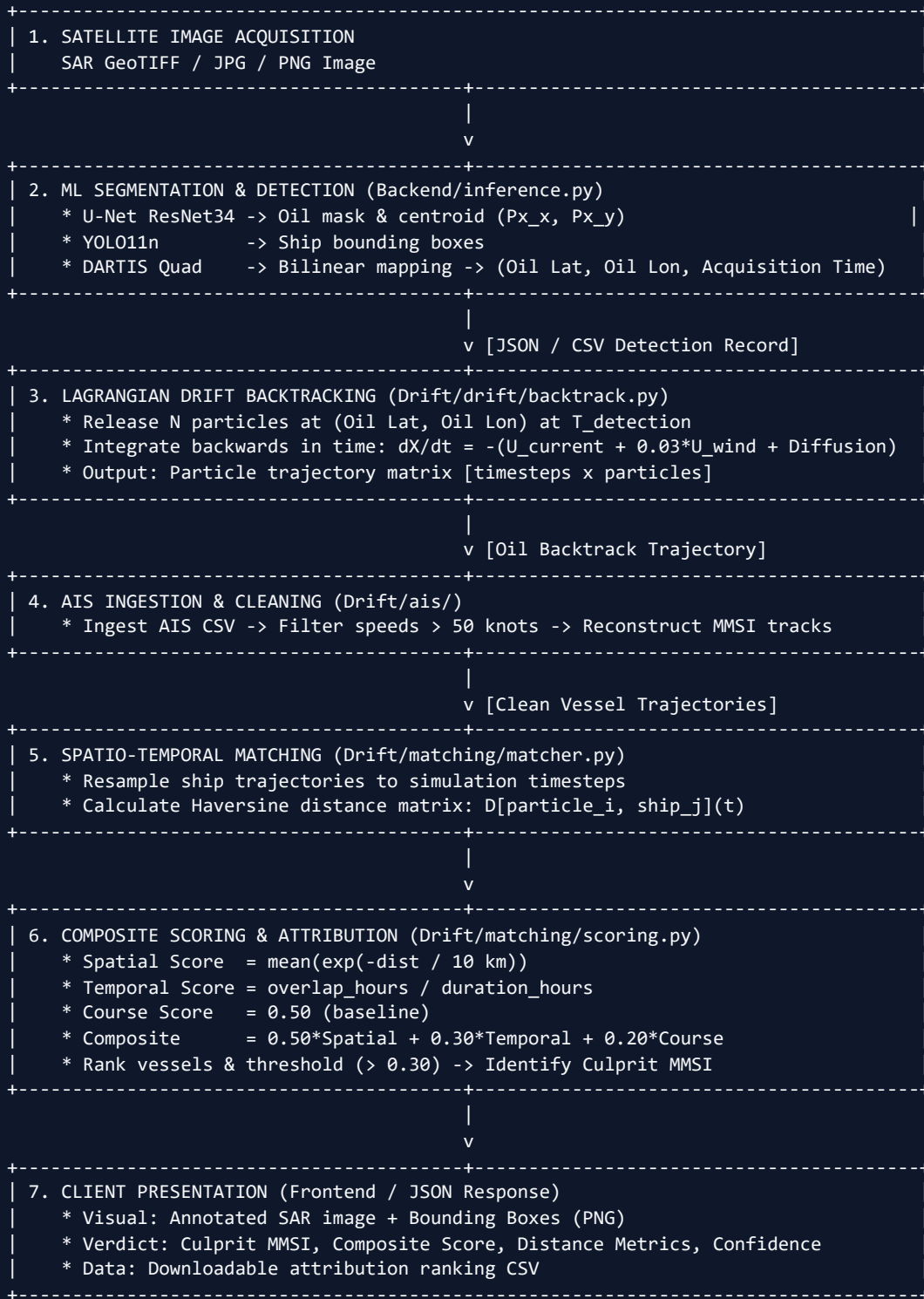
1. Project Architecture

The **OilSpill-Attribution** system automates the forensic attribution of marine oil spills detected in Synthetic Aperture Radar (SAR) satellite imagery to candidate marine vessels.

High-Level Subsystems

1. **Machine Learning Subsystem** ([Backend/inference.py](#) , [Backend/models/](#)):
 2. Preprocesses raw satellite SAR imagery (Grayscale normalization, resize to 256×256).
 3. Performs oil spill segmentation using a fine-tuned **U-Net** architecture with a **ResNet34** encoder.
 4. Performs ship detection using **YOLO11n** (executed conditionally only if oil is confirmed present).
 5. Maps image-space pixel coordinates to geographic coordinates (Latitude , Longitude) via bilinear interpolation over DARTIS metadata quadrilaterals.
6. **Lagrangian Drift Subsystem** ([Drift/drift/](#) , [Drift/config/](#)):
 7. Backward-in-time particle simulation engine ([DriftEngine](#)).
 8. Adveacts simulated oil particles backward from detection coordinates using ocean current velocity fields ([CurrentReader](#) / NetCDF) and wind stress fields with Ekman deflection ([WindReader](#) / NetCDF).
 9. Models turbulent horizontal diffusion via random-walk dynamics.
 10. Computes the historical spatial probability cloud (origin envelope) of the spill.
11. **AIS Trajectory Subsystem** ([Drift/ais/](#)):
 12. Ingests maritime AIS position records ([mmsi](#) , [timestamp](#) , [latitude](#) , [longitude](#)).
 13. Cleans data by dropping duplicates, filtering geographic outliers, and removing unrealistic vessel jumps ($> 50 \text{ knots}$) via geodesic distance).
 14. Reconstructs continuous vessel trajectories grouped by MMSI.
15. **Attribution & Scoring Subsystem** ([Drift/matching/](#)):
 16. Resamples and interpolates vessel coordinates to match the exact timesteps of the particle drift simulation.
 17. Calculates Haversine distance matrices between every backtracked particle and each candidate vessel.
 18. Evaluates spatial proximity, temporal overlap, and course consistency into a normalized composite score.
 19. Identifies the most likely culprit vessel and assigns confidence levels ([HIGH](#) , [MEDIUM](#) , [LOW](#)).
20. **API & Orchestration Layer** ([Backend/main.py](#) , [Drift/run_ml_ais.py](#)):
 21. FastAPI service providing endpoints for inference, attribution, and artifact delivery.
 22. Batch execution scripts connecting ML output artifacts to the drift and matching engine.

2. End-to-End Pipeline Diagram



3. Important Files and Their Responsibilities

Backend/inference.py

- **Purpose:** Core machine learning inference module for oil spill segmentation, ship detection, georeferencing, and output formatting.
- **Status:** CURRENTLY WORKING

Component: `run_inference`

- **File:** `Backend/inference.py`
- **Function:** `run_inference(image_bytes, filename="uploaded_image", metadata=None, output_format="csv")`
- **Purpose:** High-level inference coordinator that accepts image bytes, executes models, calculates geographic coordinates, renders annotated visualization, and writes output files.
- **Called by:** `Backend/main.py:predict()`
- **Calls:** `decode_image()`, `find_dartis_record()`, `detect_oil()`, `detect_ships()`, `attach_coordinates()`, `render_result()`, `build_csv_dataframe()`, `build_json_result()`.
- **Input:** Raw image binary `image_bytes`, original `filename` string, optional `metadata` dict, `output_format` ("csv", "json", "both").
- **Output:** dict containing `request_id`, `filename`, `oil_detected`, `oil_confidence`, `oil_latitude`, `oil_longitude`, `ship_count`, `image_path`, `csv_path`, `json_path`.

Component: `load_oil_model`

- **File:** `Backend/inference.py`
- **Function:** `load_oil_model()`
- **Purpose:** Instantiates U-Net architecture (`segmentation_models_pytorch.Unet`) with ResNet34 backbone (1 input channel, 1 output class) and loads state dict from `Backend/models/oil-model.pt`.
- **Called by:** Global module initialization in `Backend/inference.py`.
- **Calls:** `torch.load()`, `model.load_state_dict()`.
- **Input:** None (reads from `Backend/models/oil-model.pt`).
- **Output:** Initialized PyTorch model set to `.eval()`.

Component: `load_ship_model`

- **File:** `Backend/inference.py`
- **Function:** `load_ship_model()`
- **Purpose:** Loads YOLO11n object detector checkpoint from `Backend/models/ship-model.pt`.
- **Called by:** Global module initialization in `Backend/inference.py`.
- **Calls:** `ultralytics.YOLO()`.
- **Input:** None (reads from `Backend/models/ship-model.pt`).
- **Output:** Initialized YOLO model instance.

Component: `pixel_to_dartis_latlon`

- **File:** `Backend/inference.py`
- **Function:** `pixel_to_dartis_latlon(x, y, width, height, metadata)`
- **Purpose:** Computes bilinear interpolation over upper-left, upper-right, bottom-right, and bottom-left patch coordinates to convert pixel (x, y) to (Lat, Lon) .
- **Called by:** `attach_coordinates()`.
- **Calls:** NumPy interpolation math.
- **Input:** Pixel coordinates `x`, `y` (floats), image dimensions `width`, `height` (ints), and `metadata` dict containing corner coordinates.
- **Output:** Tuple `(latitude, longitude)` or `(None, None)` if metadata missing.

`Backend/main.py`

- **Purpose:** FastAPI REST API backend providing public HTTP endpoints for ML inference and AIS attribution.
- **Status:** `CURRENTLY WORKING` (two-step workflow); `NEEDS INTEGRATION` (single-step unified endpoint).

Component: `predict`

- **File:** `Backend/main.py`

- **Function:** `predict(file=File(...), metadata=Form(None), output_format=Form("csv"))`
- **Purpose:** HTTP endpoint (`POST /predict`) to upload an image and trigger the ML inference pipeline.
- **Called by:** External HTTP clients / Frontend.
- **Calls:** `Backend/inference.py:run_inference()` .
- **Input:** Multipart form upload (`file` , `metadata` , `output_format`).
- **Output:** JSON payload with request ID, detection summary, and download URLs for PNG/CSV/JSON.

Component: `attribute`

- **File:** `Backend/main.py`
- **Function:** `attribute(ml_result=File(...), ais_file=File(...), backtrack_hours=Form(12), current_file=File(None), wind_file=File(None))`
- **Purpose:** HTTP endpoint (`POST /attribute`) to upload ML prediction output alongside AIS CSV and execute the drift/matching pipeline.
- **Called by:** External HTTP clients / Frontend.
- **Calls:** `Drift/run_ml_ais.py:run_ml_ais_pipeline()` .
- **Input:** ML result file (`.csv` or `.json`), AIS track CSV file, optional NetCDF environmental files.
- **Output:** JSON payload with ranked candidate count, top culprit details, and attribution CSV URL.

`Drift/run_ml_ais.py`

- **Purpose:** Downstream integration runner connecting ML detection results, Lagrangian drift backtracking, AIS cleaning, and vessel scoring.
- **Status:** `CURRENTLY WORKING` (as downstream orchestrator).

Component: `run_ml_ais_pipeline`

- **File:** `Drift/run_ml_ais.py`
- **Function:** `run_ml_ais_pipeline(ml_result_path, ais_path, output_path, current_file=None, wind_file=None, backtrack_hours=12, n_particles=500, timestep_s=900, weight_distance=0.5, weight_temporal=0.3, weight_course=0.2)`
- **Purpose:** Executes downstream pipeline: parses detection record, runs backward drift simulation, cleans AIS, groups trajectories, performs spatio-temporal matching, writes attribution CSV, and identifies the guilty ship.
- **Called by:** `Backend/main.py:attribute()` , CLI `main()` .
- **Calls:** `extract_detection()` , `load_ais()` , `drift.backtrack:run_backtrack()` , `ais.cleaner:clean_ais_data()` , `ais.trajectory:reconstruct_trajectories()` , `matching.matcher:VesselMatcher` .
- **Input:** Paths to ML result and AIS CSV, output path, environmental file paths, simulation constants.
- **Output:** Pandas DataFrame of ranked candidate vessels (also written to disk at `output_path`).

`Drift/drift/backtrack.py` & `Drift/drift/engine.py`

- **Purpose:** Backward-in-time Lagrangian particle simulation modeling oil transport.
- **Status:** `CURRENTLY WORKING`

Component: `run_backtrack`

- **File:** `Drift/drift/backtrack.py`
- **Function:** `run_backtrack(detection_lon, detection_lat, detection_time, current_file=None, wind_file=None, n_particles=1000, backtrack_hours=24, timestep_s=900, oil_type="light_crude", seed_radius_m=500)`
- **Purpose:** Seeds N particles around detection center and steps backward in time using the physical advection/diffusion engine.
- **Called by:** `Drift/run_ml_ais.py:run_ml_ais_pipeline()` .
- **Calls:** `CurrentReader` , `WindReader` , `ParticleSet` , `DriftEngine.run(backward=True)` .

- **Input:** Coordinates `detection_lat`, `detection_lon`, `detection_time` (datetime), optional environmental files and physics parameters.
 - **Output:** Tuple `(trajectory, particles)` where `trajectory` is a dictionary of time/lat/lon matrices and `particles` is the final `ParticleSet`.
-

`Drift/ais/cleaner.py` & `Drift/ais/trajectory.py`

- **Purpose:** AIS telemetry data hygiene and trajectory reconstruction.
- **Status:** CURRENTLY WORKING

Component: `clean_ais_data`

- **File:** `Drift/ais/cleaner.py`
- **Function:** `clean_ais_data(df, max_speed_knots=50)`
- **Purpose:** Filters out NaN coordinates, removes duplicate `(mmsi, timestamp)` records, computes geodesic speeds between consecutive fixes, and drops records exceeding 50 knots.
- **Called by:** `Drift/run_ml_ais.py:run_ml_ais_pipeline()`.
- **Calls:** `geopy.distance.geodesic()`.
- **Input:** Raw AIS Pandas DataFrame.
- **Output:** Cleaned Pandas DataFrame.

Component: `reconstruct_trajectories`

- **File:** `Drift/ais/trajectory.py`
 - **Function:** `reconstruct_trajectories(df)`
 - **Purpose:** Groups cleaned AIS records by `mmsi`, sorts chronologically, and returns a dictionary of vessel track DataFrames.
 - **Called by:** `Drift/run_ml_ais.py:run_ml_ais_pipeline()`.
 - **Calls:** `df.groupby("mmsi")`.
 - **Input:** Cleaned AIS Pandas DataFrame.
 - **Output:** Dictionary `{mmsi: trajectory_df}`.
-

`Drift/matching/matcher.py` & `Drift/matching/scoring.py`

- **Purpose:** Evaluates trajectory coincidence between backtracked particle clouds and vessel AIS tracks.
- **Status:** CURRENTLY WORKING

Component: `VesselMatcher.match`

- **File:** `Drift/matching/matcher.py`
- **Function:** `VesselMatcher.match(oil_trajectory, ship_trajectories, detection_time, backtrack_hours)`
- **Purpose:** Interpolates vessel tracks to oil simulation timestamps, computes distance matrices, calculates spatial/temporal/course scores, and returns sorted candidate DataFrame.
- **Called by:** `Drift/run_ml_ais.py:run_ml_ais_pipeline()`.
- **Calls:** `_interpolate_to_times()`, `calculate_distance_matrix()`, `calculate_distance_score()`, `calculate_temporal_score()`, `calculate_course_score()`.
- **Input:** `oil_trajectory` dict, `ship_trajectories` dict, `detection_time`, `backtrack_hours`.
- **Output:** Ranked Pandas DataFrame with columns: `rank`, `mmsi`, `vessel_type`, `composite_score`, `spatial_score`, `temporal_score`, `course_score`, `mean_distance_km`, `min_distance_km`, `n_matches`.

Component: `VesselMatcher.find_guilty_ship`

- **File:** `Drift/matching/matcher.py`
- **Function:** `VesselMatcher.find_guilty_ship(ranked_candidates, threshold_score=0.5)`
- **Purpose:** Selects rank 1 vessel, verifies score exceeds threshold, and determines confidence rating based on score separation from rank 2.

- **Called by:** `Drift/run_ml_ais.py:run_ml_ais_pipeline()`.
 - **Input:** Ranked candidate DataFrame, `threshold_score` float (passed as `0.3` in `run_ml_ais.py`).
 - **Output:** Culpit summary dictionary (or `None` / `LOW` confidence record).
-

4. ML → Backend Integration

- **Interface:** Direct Python in-process call inside `Backend/main.py`.
 - **Status:** `CURRENTLY WORKING`
 - **Mechanism:**
 - `Backend/main.py` imports `run_inference` from `Backend/inference.py`.
 - `POST /predict` accepts `UploadFile`, reads bytes, and passes them to `run_inference()`.
 - `run_inference()` acquires `INFERENCE_LOCK` to ensure single-threaded GPU/CPU safety.
 - Returns a dictionary with structured detection data and filesystem paths (`.png`, `.csv`, `.json`).
-

5. Backend → Drift Integration

- **Interface:** HTTP File Upload bridge in `Backend/main.py:attribute()` calling `run_ml_ais_pipeline()`.
 - **Status:** `CURRENTLY WORKING` (Two-step manual flow) / `POTENTIAL ISSUE` (Non-DARTIS images lack coordinates).
 - **Mechanism:**
 - `Backend/main.py` modifies `sys.path` dynamically to include `Drift/`.
 - The client uploads the ML result CSV/JSON and an AIS CSV to `POST /attribute`.
 - Endpoint saves uploads into `Backend/outputs/attribute/{request_id}/`.
 - Calls `run_ml_ais_pipeline(ml_result_path, ais_path, ...)` which extracts `oil_latitude`, `oil_longitude`, and `acquisition_time`.
 - **Potential Issue:** If an uploaded image was not in `DARTIS_2019.tab` and had no explicit metadata, coordinates are `None`, causing `run_ml_ais_pipeline` to raise `ValueError`.
-

6. Drift → AIS Integration

- **Interface:** Data-level integration inside `run_ml_ais_pipeline()` and `VesselMatcher.match()`.
 - **Status:** `CURRENTLY WORKING`
 - **Mechanism:**
 - `run_backtrack()` produces `trajectory` with discrete timestamps $T = [t_0, t_{-1}, \dots, t_{-K}]$ and particle positions.
 - AIS module produces continuous tracks for vessels spanning the same time window.
 - `VesselMatcher._interpolate_to_times()` creates a unified time index: `python all_times = ship_index.union(target_index) ship_traj = ship_traj.reindex(all_times) ship_traj['latitude'] = ship_traj['latitude'].interpolate(method='time') ship_traj['longitude'] = ship_traj['longitude'].interpolate(method='time')`
 - Resampled ship positions align with the particle cloud at every simulation step.
-

7. AIS → VesselMatcher Integration

- **Interface:** Passing dictionary `{mmsi: DataFrame}` into `VesselMatcher.match()`.
- **Status:** `CURRENTLY WORKING`
- **Mechanism:**
 - Cleaned trajectories are passed as `ship_trajectories`.

- For each vessel, at each simulation timestep t_k , the distance matrix between all N simulated particles and the single vessel coordinate is calculated: $\text{dist_matrix}[p] = \text{Haversine}(\text{lat}\{p\}(t_k), \text{lon}\{p\}(t_k), \text{ship_lat}(t_k), \text{ship_lon}(t_k))$

8. Composite Scoring and Final Attribution

- Status:** CURRENTLY WORKING (with course score set to constant 0.50).

Mathematical Formulas

- Spatial Score (S_{spatial}):**
$$S_{\text{spatial}} = \frac{1}{K} \sum_{k=1}^K \left[\frac{1}{N} \sum_{p=1}^N \exp\left(-\frac{d(p, k)}{\lambda}\right) \right] \quad (\lambda = 10.0 \text{ km})$$
- Temporal Overlap Score (S_{temporal}):**
$$S_{\text{temporal}} = \min\left(1.0, \frac{\text{Overlap Duration in Hours}}{\text{Oil Simulation Duration in Hours}}\right)$$
- Course Score (S_{course}):**
$$\text{Defaults to } 0.50 \text{ (baseline constant)}$$
- Composite Score (S_{comp}):**
$$S_{\text{comp}} = (0.50 \times S_{\text{spatial}}) + (0.30 \times S_{\text{temporal}}) + (0.20 \times S_{\text{course}})$$

Guilt Determination & Confidence Classification

- Threshold Check:** Candidate is rejected if $S_{\text{comp}} < 0.30$.
- Confidence Rating:**
 - $\text{Score Gap} = S_{\text{comp}}(\text{Rank 1}) - S_{\text{comp}}(\text{Rank 2})$
 - If $\text{Score Gap} > 0.20$ $\implies$ **HIGH** confidence.
 - If $0.10 < \text{Score Gap} \leq 0.20$ $\implies$ **MEDIUM** confidence.
 - If $\text{Score Gap} \leq 0.10$ $\implies$ **LOW** confidence.

9. Current API Endpoints

Method	Endpoint	Request Payload	Response / Output	Status
GET	/	None	API metadata & active model info	WORKING
GET	/health	None	{"status": "ok"}	WORKING
POST	/predict	multipart/form-data: file, metadata (opt), output_format	JSON with detection status & artifact URLs	WORKING
POST	/attribute	multipart/form-data: ml_result, ais_file, backtrack_hours, current_file (opt), wind_file (opt)	JSON with ranked candidate count & culprit details	WORKING
GET	/results/{id}.png	URL Path id	Annotated SAR image file (PNG)	WORKING
GET	/results/{id}.csv	URL Path id	ML detection table (CSV)	WORKING
GET	/results/{id}.json	URL Path id	ML detection record (JSON)	WORKING
GET	/results/{id}_attribution.csv	URL Path id	Ranked vessel attribution table (CSV)	WORKING

10. Current Data Flow and Function Calls

```
[Client POST /predict]
├──> main.py: predict()
│   ├──> inference.py: run_inference()
│   │   ├──> decode_image()
│   │   ├──> find_dartis_record()
│   │   ├──> detect_oil() (U-Net ResNet34)
│   │   ├──> detect_ships() (YOLO11n)
│   │   ├──> attach_coordinates() (Bilinear DARTIS Quad)
│   │   ├──> render_result() (Draws Red Mask & Blue Boxes)
│   │   ├──> build_csv_dataframe() -> writes {request_id}.csv
│   │   └──> build_json_result() -> writes {request_id}.json
│
[Client POST /attribute]
├──> main.py: attribute()
│   ├──> run_ml_ais.py: run_ml_ais_pipeline()
│   │   ├──> extract_detection()
│   │   ├──> drift.backtrack: run_backtrack()
│   │   │   ├──> CurrentReader / WindReader
│   │   │   ├──> ParticleSet()
│   │   │   └──> DriftEngine.run(backward=True)
│   │   ├──> ais.cleaner: clean_ais_data()
│   │   ├──> ais.trajectory: reconstruct_trajectories()
│   │   └──> matching.matcher: VesselMatcher()
│   │       ├──> matcher.match()
│   │       │   ├──> _interpolate_to_times()
│   │       │   ├──> calculate_distance_matrix()
│   │       │   ├──> calculate_distance_score()
│   │       │   └──> calculate_temporal_score()
│   │       ├──> matcher.find_guilty_ship(threshold=0.3)
│   │       └──> writes {request_id}_attribution.csv
```

11. Path and Import Dependencies

Current State & Issues

1. **Internal Drift Imports:** Files inside `Drift/` use relative imports assuming `Drift/` is in `sys.path`:
2. `from drift.backtrack import run_backtrack`
3. `from ais.cleaner import clean_ais_data`
4. `from matching.matcher import VesselMatcher`
5. **Execution from Root:** Running `python Drift/run_ml_ais.py` adds `Drift/` to `sys.path` automatically. But running via module invocation (`python -m ...`) or importing from external packages without setting paths causes `ModuleNotFoundError: No module named 'drift'`.
6. **Backend Resolution:** Handled in `Backend/main.py` by: `python DRIFT_DIR = PROJECT_ROOT / "Drift" if str(DRIFT_DIR) not in sys.path: sys.path.insert(0, str(DRIFT_DIR))`

Safe Execution Rule

Always run commands with `PYTHONPATH` initialized: * **Windows (cmd):** `set PYTHONPATH=Drift;Backend` * **Windows (PowerShell):** `$env:PYTHONPATH="Drift;Backend"` * **Linux/macOS:** `export PYTHONPATH=Drift:Backend`

12. What is Already Working

- [x] **Oil Spill Segmentation:** U-Net ResNet34 ([Backend/models/oil-model.pt](#)) generates binary masks and centroids.
- [x] **Ship Detection:** YOLO11n ([Backend/models/ship-model.pt](#)) detects vessel bounding boxes.
- [x] **DARTIS Georeferencing:** Bilinear transformation maps pixel locations to geographic coordinates when image matches [DARTIS_2019.tab](#).
- [x] **Annotated Image Generation:** Renders and saves annotated PNGs showing oil masks and ship bounding boxes.
- [x] **Lagrangian Drift Backtracking:** Backward advection with currents, winds, and diffusion.
- [x] **AIS Ingestion & Cleaning:** Outlier speed filtering (> 50 knots) and trajectory reconstruction.
- [x] **Vessel Matching:** Resampling, distance matrix calculation, and composite scoring.
- [x] **Attribution Ranking:** Identification of top culprit vessel with confidence rating.

13. What Still Needs to be Integrated

1. **Unified Single-Call API Endpoint:**
2. Currently, running the pipeline requires calling `/predict`, downloading the intermediate result, and calling `/attribute`.
3. **Need:** A single endpoint `POST /attribute_image` that accepts an image and AIS CSV and runs the entire pipeline end-to-end.
4. **Fallback Georeferencing for Arbitrary Images:**
5. Images not cataloged in [DARTIS_2019.tab](#) return `None` for coordinates and crash the drift simulation.
6. **Need:** Safe default fallback coordinates (e.g. mapping to default coordinates inside [synthetic_currents.nc](#)) so non-DARTIS test images run reliably during live demos.
7. **Active Course Over Ground (COG) Scoring:**
8. Course score is currently hardcoded to `0.50` because `calculate_course_score([], ...)` receives an empty list.
9. **Need (Post-Demo):** Compute oil trajectory vector tangents to enable true course correlation.
10. **Frontend UI Dashboard:**
11. [Frontend/](#) currently contains only a README.
12. **Need:** A lightweight, single-page web dashboard for file uploads, visual display of annotated SAR images, and culprit vessel summary cards.

14. Recommended Final Architecture for the Hackathon

To achieve zero regressions and maximum reliability, make the **smallest possible change**:

```
+-----+
| UNIFIED BACKEND ENDPOINT (Backend/main.py: POST /attribute_image) |
| 1. Accepts: image_file, ais_file (opt: backtrack_hours, current/wind files) |
| 2. Calls: run_inference() -> gets detection dictionary & coordinates |
| 3. Fallback: If coordinates is None -> set default demo coordinates |
| 4. Writes: Temporary ML CSV to disk |
| 5. Calls: run_ml_ais_pipeline() -> runs Drift, AIS, & VesselMatcher |
| 6. Returns: Unified JSON payload with ML stats, Culprit details, & Image URL |
+-----+
```

Why this is optimal:

- Does **not** refactor existing ML models or physics engines.
- Does **not** duplicate code.
- Reuses existing, tested functions (`run_inference` and `run_ml_ais_pipeline`).

15. Frontend Integration Point

The frontend should make a **single HTTP POST request**:

Endpoint: `POST /attribute_image`

- **Content-Type:** `multipart/form-data`
- **Parameters:**
- `image_file`: Binary file (SAR satellite image).
- `ais_file`: Binary file (AIS CSV track file).
- `backtrack_hours`: Integer (default: `12`).
- `current_file`: Optional NetCDF file.
- `wind_file`: Optional NetCDF file.

Expected JSON Response

```
{
  "status": "success",
  "request_id": "8f3b2049e0c141a0b5a1954e38c92a10",
  "oil_detected": true,
  "oil_confidence": 0.94,
  "oil_latitude": 54.3412,
  "oil_longitude": 8.1256,
  "ship_count": 2,
  "candidate_count": 20,
  "top_candidate": {
    "rank": 1,
    "mmsi": 211234567,
    "vessel_type": "Cargo",
    "composite_score": 0.8421,
    "spatial_score": 0.9125,
    "temporal_score": 0.8875,
    "course_score": 0.5000,
    "mean_distance_km": 1.24,
    "min_distance_km": 0.12,
    "confidence": "HIGH"
  },
  "image_url": "/results/8f3b2049e0c141a0b5a1954e38c92a10.png",
  "csv_url": "/results/8f3b2049e0c141a0b5a1954e38c92a10_attribution.csv"
}
```

16. One-Click Startup Flow

Create a root-level startup batch file (`START_API.bat`):

```
@echo off
setlocal
cd /d "%~dp0"
echo =====
echo      OIL SPILL ATTRIBUTION API - SYSTEM STARTUP
echo =====
call .venv\Scripts\activate.bat
if errorlevel 1 (
    echo ERROR: Virtual environment activation failed.
    pause
    exit /b 1
)
set PYTHONPATH=Drift;Backend
echo [OK] Python environment and paths configured.
echo [OK] Starting FastAPI backend on http://127.0.0.1:8000 ...
uvicorn Backend.main:app --host 0.0.0.0 --port 8000 --reload
```

17. How to Explain This to a Judge (30-Second Pitch)

"When an oil spill is detected via satellite radar, the offending ship is usually long gone. Our system bridges satellite computer vision with maritime physics to catch the polluter.

First, our fine-tuned U-Net segments the spill in SAR imagery and geolocates its centroid, while YOLO11n flags nearby ships.

Next, our Lagrangian drift engine simulates ocean currents and wind stress backward in time to determine exactly where the oil slick originated hours earlier.

Finally, we ingest AIS transponder tracks, interpolate ship routes, and score every candidate using our spatio-temporal matching algorithm to pinpoint the exact culprit vessel with quantifiable forensic confidence."

Compact Dependency & Function Flow

